

Program Efficiency and Complexity

Part-2 (More Examples)

Questions that will be answered

- What is a “good” or "efficient" program?
- How to measure the efficiency of a program?
- How to analyse a simple program?
- How to compare different programs?
- What is the big-O notation?
- What is the impact of input on program performance?
- What are the standard program analysis techniques?
- Do we need fast machines or fast algorithms?

Key Topics:

- Introduction
- Generalizing Running Time
- Doing a Timing Analysis
- Big-Oh Notation
- Big-Oh Operations
- Analyzing Some Simple Programs – no Subprogram calls
- Worst-Case and Average Case Analysis
- Analyzing Programs with Non-Recursive Subprogram Calls
- Classes of Problems

Why Analyze Algorithms?

An algorithm can be analyzed in terms of time efficiency or space utilization. We will consider only the former right now. The running time of an algorithm is influenced by several factors:

- Speed of the machine running the program
- Language in which the program was written. For example, programs written in assembly language generally run faster than those written in C or C++, which in turn tend to run faster than those written in Java.
- Efficiency of the compiler that created the program
- The size of the input: processing 1000 records will take more time than processing 10 records.
- Organization of the input: if the item we are searching for is at the top of the list, it will take less time to find it than if it is at the bottom.

Which is better?

The running time of a program.

Program easy to understand?

Program easy to code and debug?

Program making efficient use of resources?

Program running as fast as possible?

Measuring Efficiency

Ways of measuring efficiency:

- Run the program and see how long it takes
- Run the program and see how much memory it uses

•Lots of variables to control:

- What is the input data?
- What is the hardware platform?
- What is the programming language/compiler?
- Just because one program is faster than another right now, means it will always be faster?

Measuring Efficiency

Want to achieve platform-independence

- Use an abstract machine that uses *steps* of time and *units* of memory, instead of seconds or bytes
 - - each elementary operation takes 1 step
 - - each elementary instance occupies 1 unit of memory

A Simple Example

```
// Input: int A[N], array of N integers  
// Output: Sum of all numbers in array  
A
```

```
int Sum(int A[], int N) {  
    int s=0;  
    for (int i=0; i< N; i++)  
        s = s + A[i];  
    return s;  
}
```

How should we analyse this?

A Simple Example

Analysis of Sum

1.) Describe the size of the input in terms of one or more parameters:

- Input to Sum is an array of N ints, so size is N .

2.) Then, count how many steps are used for an input of that size:

- A step is an elementary operation such as
 $+$, $<$, $=$, $A[i]$

A Simple Example

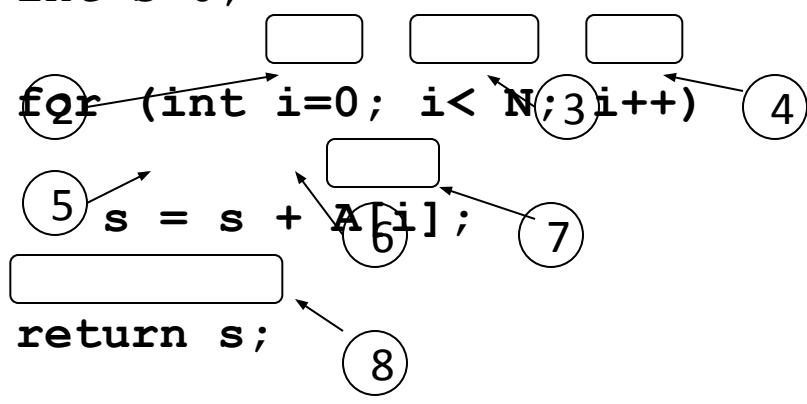
Analysis of Sum (2)

// Input: int A[N], array of N integers

// Output: Sum of all numbers in array

A

```
int Sum(int A[], int N {  
    int s=0;  
    for (int i=0; i< N; i++)  
        s = s + A[i];  
    return s;  
}
```



1,2,8: Once

3,4,5,6,7: Once per each iteration
of for loop, N iteration

Total: $5N + 3$

The *complexity function* of the
algorithm is : $f(N) = 5N + 3$

Complexity

- Complexity is a function $T(n)$ which measures the time or space used by an algorithm with respect to the input size n .
- The running Time of an algorithm on a particular input is determined by the number of “**Elementary Operations**” executed.

Complexity and Input Size

- Complexity function that express relationship between time and input size is usually much more complex
- We are not so much interested in the time and space complexity for small inputs rather the function is only calculated for large inputs.
- Example?

Analysis: A Simple Example

How $5N+3$ Grows

Estimated running time for different values of N :

$N = 10 \quad \Rightarrow 53 \text{ steps}$

$N = 100 \quad \Rightarrow 503 \text{ steps}$

$N = 1,000 \quad \Rightarrow 5003 \text{ steps}$

$N = 1,000,000 \quad \Rightarrow 5,000,003 \text{ steps}$

As N grows, the number of steps grow in *linear* proportion to N for this Sum function.

Analysis: A Simple Example

What Dominates?

What about the 5 in $5N+3$? What about the $+3$?

- As N gets large, the $+3$ becomes insignificant
- 5 is inaccurate, as different operations require varying amounts of time

What is fundamental is that the time is *linear* in N .

Asymptotic Complexity: As N gets large, concentrate on the highest order term:

- Drop lower order terms such as $+3$
- Drop the constant coefficient of the highest order term i.e. N

Complexity-Growth of Function

- The **growth of the complexity functions** is what is more important for the analysis.
- The growth of time and space complexity with increasing input size n is a suitable measure for the comparison of algorithms.

Analyzing Running Time

$T(n)$, or the running time of a particular algorithm on input of size n , is taken to be the number of times the instructions in the algorithm are executed. Pseudo code algorithm illustrates the calculation of the mean (average) of a set of n numbers:

1. n = read input from user
2. $sum = 0$
3. $i = 0$
4. while $i < n$
5. $number$ = read input from user
6. $sum = sum + number$
7. $i = i + 1$
8. $mean = sum / n$

The computing time for this algorithm in terms on input size n is: $T(n) = 4n + 5$.

<u>Statement</u>	<u>Number of times executed</u>
1	1
2	1
3	1
4	$n+1$
5	n
6	n
7	n
8	1

Analysis: A Simple Example

Asymptotic Complexity

- The $5N+3$ time bound is said to "grow asymptotically" like N
- This gives us an approximation of the complexity of the algorithm
- Ignores lots of (machine dependent) details, concentrate on the bigger picture

Big-Oh Notation

Definition 1: Let $f(n)$ and $g(n)$ be two functions. We write:

$$f(n) = O(g(n)) \quad \text{or} \quad f = O(g)$$

(read " f of n is big oh of g of n " or " f is big oh of g ") if there is a positive integer C such that $f(n) \leq C * g(n)$ for all positive integers n .

The basic idea of big-Oh notation is this: Suppose f and g are both real-valued functions of a real variable n . If, for large values of n , the graph of f lies closer to the horizontal axis than the graph of some multiple of g , then f is of order g , i.e., $f(n) = O(g(n))$. So, $g(n)$ represents an *upper bound* on $f(n)$.

Example 1

Suppose $f(n) = 5n$ and $g(n) = n$.

- To show that $f = O(g)$, we have to show the existence of a constant C as given in Definition 1. Clearly 5 is such a constant so $f(n) = 5 * g(n)$.
- We could choose a larger C such as 6, because the definition states that $f(n)$ must be less than or equal to $C * g(n)$, but we usually try and find the smallest one.

Therefore, a constant C exists (we only need one) and $f = O(g)$.

Example 2

In the previous timing analysis, we ended up with $T(n) = 4n + 5$, and we concluded intuitively that $T(n) = O(n)$ because the running time grows linearly as n grows. Now, however, we can prove it mathematically:

To show that $f(n) = 4n + 5 = O(n)$, we need to produce a constant C such that:
$$f(n) \leq C * n \text{ for all } n.$$

If we try $C = 4$, this doesn't work because $4n + 5$ is not less than $4n$. We need C to be at least 9 to cover *all* n . If $n = 1$, C has to be 9, but C can be smaller for greater values of n (if $n = 100$, C can be 5). Since the chosen C must work for all n , we must use 9:

$$4n + 5 \leq 4n + 5n = 9n$$

Since we have produced a constant C that works for all n , we can conclude:

$$T(4n + 5) = O(n)$$

Example 3

Say $f(n) = n^2$: We will prove that $f(n) \neq O(n)$.

- To do this, we must show that there cannot exist a constant C that satisfies the big-Oh definition. We will prove this by contradiction.

Suppose there is a constant C that works; then, by the definition of big-Oh: $n^2 \leq C * n$ for **all** n .

- Suppose n is any positive real number greater than C , then: $n * n > C * n$, or $n^2 > C * n$.

So there exists a real number n such that $n^2 > C * n$.

This contradicts the supposition, so the supposition is false. There is no C that can work for all n .

$f(n) \neq O(n)$ when $f(n) = n^2$

Example 4

Suppose $f(n) = n^2 + 3n - 1$. We want to show that $f(n) = O(n^2)$.

$$\begin{aligned} f(n) &= n^2 + 3n - 1 \\ &< n^2 + 3n \quad (\text{subtraction makes things smaller so drop it}) \\ &\leq n^2 + 3n^2 \quad (\text{since } n \leq n^2 \text{ for all integers } n) \\ &= 4n^2 \end{aligned}$$

Therefore, if $C = 4$, we have shown that $f(n) = O(n^2)$. Notice that all we are doing is finding a simple function that is an upper bound on the original function. Because of this, we could also say that

$f(n) = O(n^3)$ since (n^3) is an upper bound on n^2

This would be a much weaker description, but it is still valid.

Example 5

Show:

$$\begin{aligned} f(n) &= 2n^7 + 6n^5 + 10n^2 - 5 = O(n^7) \\ f(n) &< 2n^7 + 6n^5 + 10n^2 \\ &\leq 2n^7 + 6n^7 + 10n^7 \\ &= 18n^7 \end{aligned}$$

thus, with $C = 18$ and we have shown that $f(n) = O(n^7)$

Any polynomial is big-Oh of its *term of highest degree*. We are also ignoring constants. Any polynomial (including a general one) can be manipulated to satisfy the big-Oh definition by doing what we did in the last example: take the absolute value of each coefficient (this can only increase the function); Then since

$$n^j \leq n^d \quad \text{if } j \leq d$$

we can change the exponents of all the terms to the highest degree (the original function must be less than this too). Finally, we add these terms together to get the largest constant C we need to find a function that is an upper bound on the original one.

Adjusting the definition of big-Oh: Many algorithms have a rate of growth that matches logarithmic functions. Recall that $\log_2 n$ is the number of times we have to divide n by 2 to get 1; or alternatively, the number of 2's we must multiply together to get n :

$$n = 2^k \quad \hat{=} \quad \log_2 n = k$$

Many "Divide and Conquer" algorithms solve a problem by dividing it into 2 smaller problems. You keep dividing until you get to the point where solving the problem is trivial. This constant division by 2 suggests a logarithmic running time.

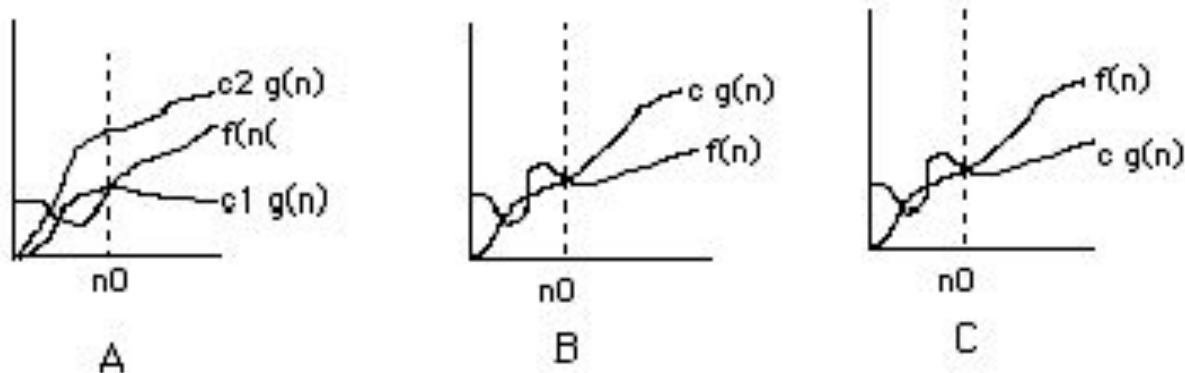
Definition 2: Let $f(n)$ and $g(n)$ be two functions. We write:

$$f(n) = O(g(n)) \quad \text{or} \quad f = O(g)$$

if there are positive integers C and N such that $f(n) \leq C * g(n)$ for all integers $n \geq N$.

Using this more general definition for big-Oh, we can now say that if we have $f(n) = 1$, then $f(n) = O(\log(n))$ since $C = 1$ and $N = 2$ will work.

With this definition, we can clearly see the difference between the three types of notation:



In all three graphs above, n_0 is the minimal possible value to get valid bounds, but any greater value will work

There is a handy theorem that relates these notations:

Theorem: For any two functions $f(n)$ and $g(n)$, $f(n) = \Theta(g(n))$ if and only if $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$.

Example 6:

Show: $f(n) = 3n^3 + 3n - 1 = \Theta(n^3)$

As implied by the theorem above, to show this result, we must show two properties:

$$f(n) = O(n^3)$$

$$f(n) = \Omega(n^3)$$

First, we show (i), using the same techniques we've already seen for big-Oh.

We consider $N = 1$, and thus we only consider $n \geq 1$ to show the big-Oh result.

$$\begin{aligned} f(n) &= 3n^3 + 3n - 1 \\ &< 3n^3 + 3n + 1 \\ &\leq 3n^3 + 3n^3 + 1n^3 \\ &= 7n^3 \end{aligned}$$

thus, with

$C = 7$ and $N = 1$ we have shown that $f(n) = O(n^3)$

Next, we show (ii). Here we must provide a lower bound for $f(n)$. Here, we choose a value for N , such that the highest order term in $f(n)$ will always dominate (be greater than) the lower order terms.

We choose $N = 2$, since for $n \geq 2$, we have $n^3 \geq 8$. This will allow n^3 to be larger than the remainder of the polynomial $(3n - 1)$ for all $n \geq 2$.

So, by subtracting an extra n^3 term, we will form a polynomial that will always be less than $f(n)$ for $n \geq 2$.

$$\begin{aligned} f(n) &= 3n^3 + 3n - 1 \\ &> 3n^3 - n^3 && \text{since } n^3 > 3n - 1 \text{ for any } n \geq 2 \\ &= 2n^3 \end{aligned}$$

Thus, with $C = 2$ and $N = 2$, we have shown that

$$f(n) = \Omega(n^3)$$

since $f(n)$ is shown to always be greater than $2n^3$.

Comparing Functions

Definition: If $f(N)$ and $g(N)$ are two complexity functions, we say

$$f(N) = O(g(N))$$

(read " $f(N)$ as order $g(N)$ ", or " $f(N)$ is big- O of $g(N)$ ")

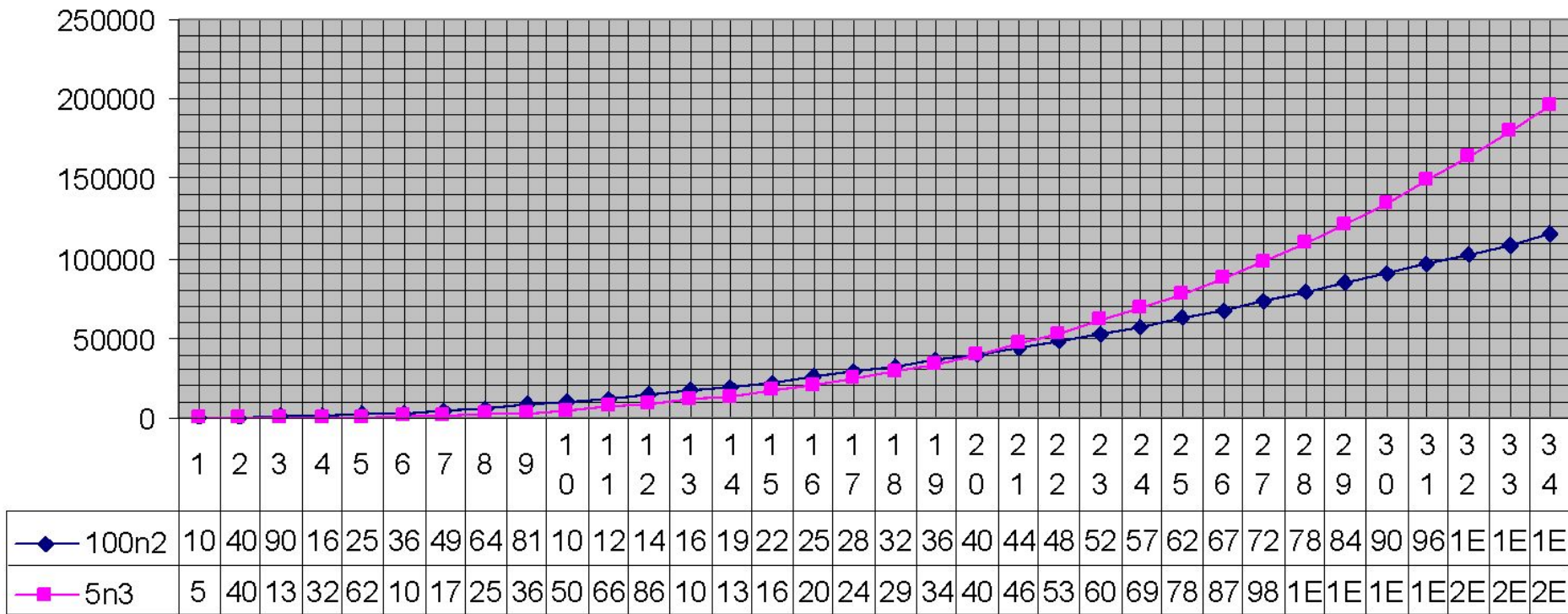
if there are constants c and N_0 such that for $N \leq N_0$, $T(N) \leq cN$

$$f(N) \leq c g(N)$$

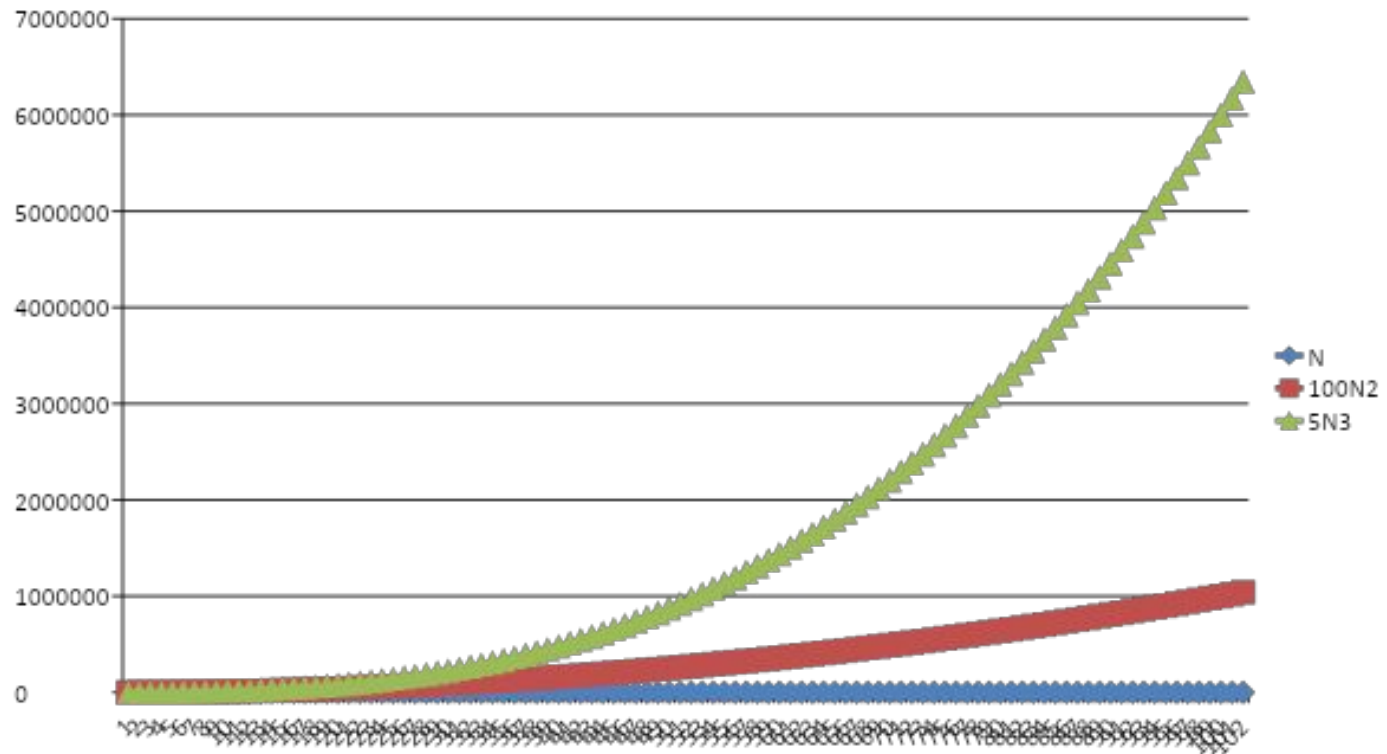
for all sufficiently large N .

Comparing Functions

$100n^2$ Vs $5n^3$, which one is better?



What is better? $100N^2$ or $5N^3$



Generalizing Running Time

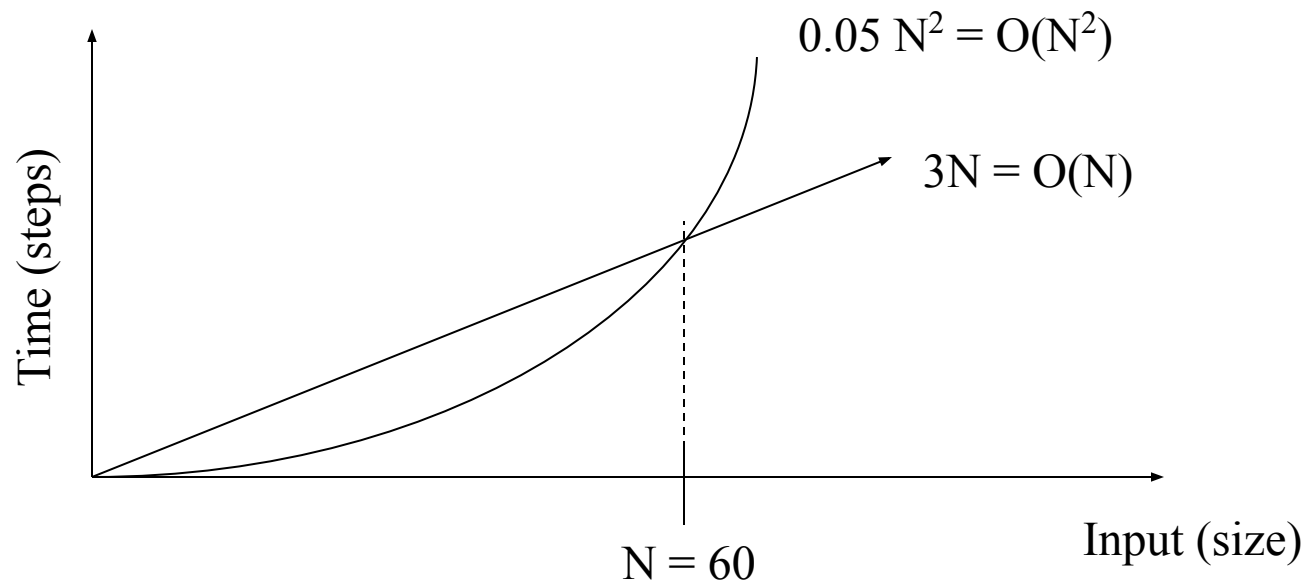
Comparing the growth of the running time as the input grows to the growth of known functions.

Input Size: n	(1)	$\log n$	n	$n \log n$	n^2	n^3	2^n
5	1	3	5	15	25	125	32
10	1	4	10	33	100	10^3	10^3
100	1	7	100	664	10^4	10^6	10^{30}
1000	1	10	1000	10^4	10^6	10^9	10^{300}
10000	1	13	10000	10^5	10^8	10^{12}	10^{3000}

Comparing Functions

Why is this useful?

As inputs get larger, any algorithm of a smaller order will be more efficient than an algorithm of a larger order



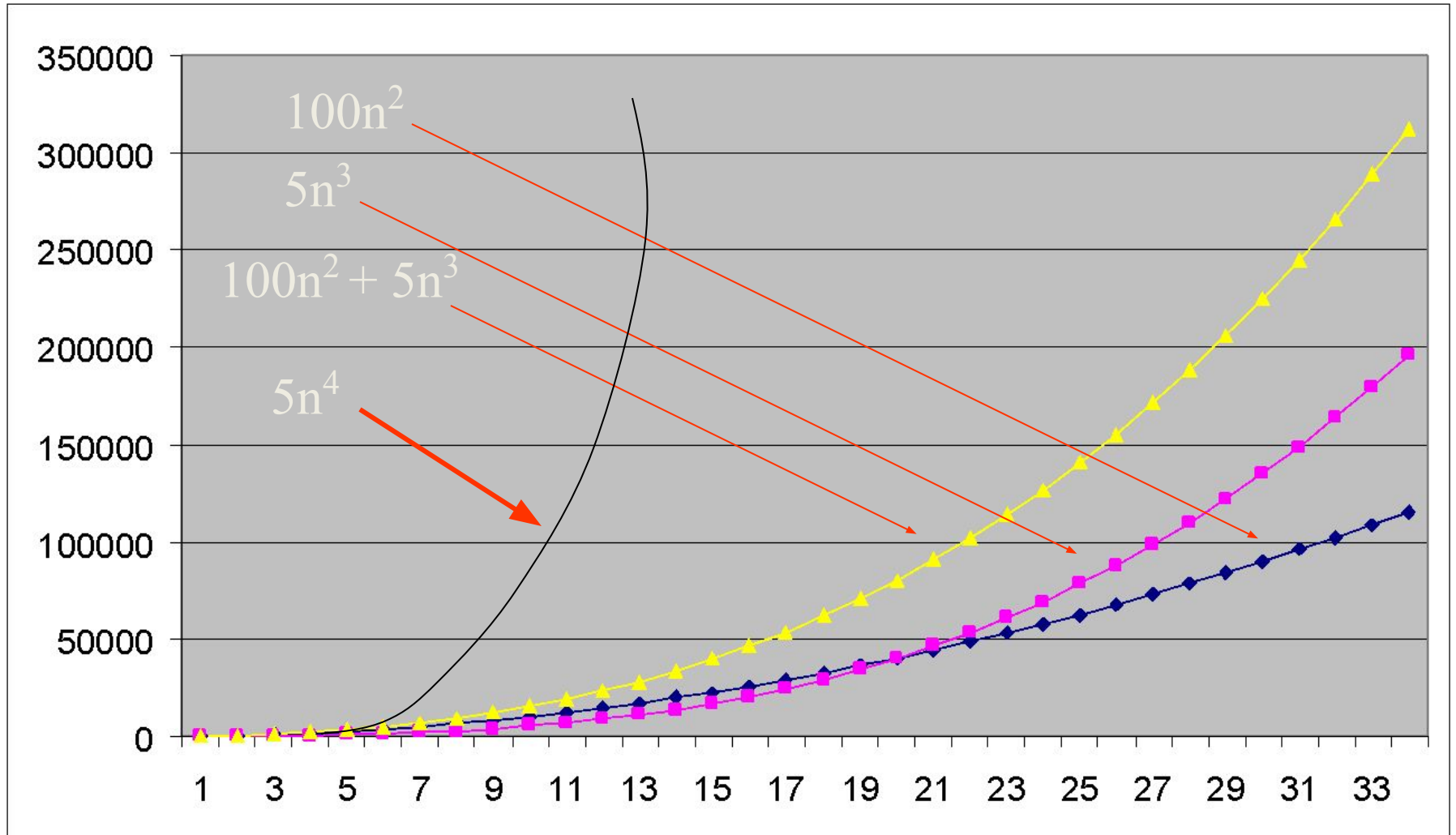
Big-O Notation

- Think of $f(N) = O(g(N))$ as
" $f(N)$ grows at most like $g(N)$ " or
" f grows no faster than g "
(ignoring constant factors, and for large N)

Important:

- Big-O is not a function!
- Never read $=$ as "equals"
- Examples:
 $5N + 3 = O(N)$
 $37N^5 + 7N^2 - 2N + 1 = O(N^5)$

Big-O Notation



Size does matter

Common Orders of Growth

Let N be the input size, and b and k be constants

$O(k) = O(1)$	Constant Time	Increasing Complexity ↓
$O(\log_b N) = O(\log N)$	Logarithmic Time	
$O(N)$	Linear Time	
$O(N \log N)$		
$O(N^2)$	Quadratic Time	
$O(N^3)$	Cubic Time	
...		
$O(k^N)$	Exponential Time	

Size does matter

What happens if we double the input size N ?

$N \log_2 N$	$5N$	$N \log_2 N$	N^2	2^N	
8	3	40	24	64	256
16	4	80	64	256	65536
32	5	160	160	1024	$\sim 10^9$
64	6	320	384	4096	$\sim 10^{19}$
128	7	640	896	16384	$\sim 10^{38}$
256	8	1280	2048	65536	$\sim 10^{76}$

Size does matter

Big Numbers

Suppose a program has run time $O(n!)$ and the run time for $n = 10$ is 1 second

For $n = 12$, the run time is 2 minutes

For $n = 14$, the run time is 6 hours

For $n = 16$, the run time is 2 months

For $n = 18$, the run time is 50 years

For $n = 20$, the run time is 200 centuries

Components of Algorithm

- Variables and values
- Instructions
- Sequences
- Selections
- Repetitions
- Procedures
- *Documentation*

Big-Oh Operations

Summation Rule

Suppose $T1(n) = O(f1(n))$ and $T2(n) = O(f2(n))$. Further, suppose that $f2$ grows no faster than $f1$, i.e., $f2(n) = O(f1(n))$. Then, we can conclude that $T1(n) + T2(n) = O(f1(n))$. More generally, the summation rule tells us $O(f1(n) + f2(n)) = O(\max(f1(n), f2(n)))$.

Proof:

Suppose that C and C' are constants such that $T1(n) \leq C * f1(n)$ and $T2(n) \leq C' * f2(n)$. Let $D =$ the larger of C and C' . Then,

$$\begin{aligned} T1(n) + T2(n) &\leq C * f1(n) + C' * f2(n) \\ &\leq D * f1(n) + D * f2(n) \\ &\leq D * (f1(n) + f2(n)) \\ &\leq O(f1(n) + f2(n)) \end{aligned}$$

Product Rule

Suppose $T1(n) = O(f1(n))$ and $T2(n) = O(f2(n))$. Then, we can conclude that $T1(n) * T2(n) = O(f1(n) * f2(n))$.

The Product Rule can be proven using a similar strategy as the Summation Rule proof.

Analyzing Some Simple Programs (with No Sub-program Calls)

General Rules:

- All basic statements (assignments, reads, writes, conditional testing, library calls) run in constant time: $O(1)$.
- The time to execute a loop is the sum, over all times around the loop, of the time to execute all the statements in the loop, plus the time to evaluate the condition for termination. Evaluation of basic termination conditions is $O(1)$ in each iteration of the loop.
- The complexity of an algorithm is determined by the complexity of the most frequently executed statements. If one set of statements have a running time of $O(n^3)$ and the rest are $O(n)$, then the complexity of the algorithm is $O(n^3)$. This is a result of the Summation Rule.

Example 7

Compute the big-Oh running time of the following C++ code segment:

```
for (i = 2; i < n; i++) {  
    sum += i;  
}
```

The number of iterations of a for loop is equal to the top index of the loop minus the bottom index, plus one more instruction to account for the final conditional test.

Note: if the for loop terminating condition is $i \leq n$, rather than $i < n$, then the number of times the conditional test is performed is:

$$((\text{top_index} + 1) - \text{bottom_index}) + 1)$$

In this case, we have $n - 2 + 1 = n - 1$. The assignment in the loop is executed $n - 2$ times. So, we have $(n - 1) + (n - 2) = (2n - 3)$ instructions executed = $O(n)$.

Example 8

Consider the sorting algorithm shown below. Find the number of instructions executed and the complexity of this algorithm.

```
1)   for (i = 1; i < n; i++) {
2)       SmallPos = i;
3)       Smallest = Array[SmallPos];
4)       for (j = i+1; j <= n; j++)
5)           if (Array[j] < Smallest) {
6)               SmallPos = j;
7)               Smallest = Array[SmallPos]
8)           }
9)       Array[SmallPos] = Array[i];
       Array[i] = Smallest;
       }
```

The total computing time is:

$$\begin{aligned} T(n) &= (n) + 4(n-1) + n(n+1)/2 - 1 + 3[n(n-1) / 2] \\ &= n + 4n - 4 + (n^2 + n)/2 - 1 + (3n^2 - 3n) / 2 \\ &= 5n - 5 + (4n^2 - 2n) / 2 \\ &= 5n - 5 + 2n^2 - n \\ &= 2n^2 + 4n - 5 \\ &= O(n^2) \end{aligned}$$

Example 9

The following program segment initializes a two-dimensional array A (which has n rows and n columns) to be an n x n identity matrix – that is, a matrix with 1's on the diagonal and 0's everywhere else. More formally, if A is an n x n identity matrix, then:

$$\mathbf{A} \times \mathbf{M} = \mathbf{M} \times \mathbf{A} = \mathbf{M}, \text{ for any } n \times n \text{ matrix } \mathbf{M}.$$

What is the complexity of this C++ code?

```
1)  cin >> n;  // Same as: n = GetInteger();
2)  for (i = 1; i <= n; i ++)
```

3) for (j = 1; j <= n; j ++)

4) A[i][j] = 0;

```
5)  for (i = 1; i <= n; i ++)
```

6) A[i][i] = 1;

Example 10

Here is a simple linear search algorithm that returns the index location of a value in an array.

```
/* a is the array of size n we are searching through */  
i = 0;  
while ((i < n) && (x != a[i]))  
    i++;  
if (i < n)  
    location = i;  
else  
    location = -1;
```

Average number of lines executed equals:

$$\begin{aligned} & 1 + 3 + 5 + \dots + (2n - 1) / n \\ &= (2(1 + 2 + 3 + \dots + n) - n) / n \end{aligned}$$

We know that $1 + 2 + 3 + \dots + n = n(n + 1) / 2$, so the average number of lines executed is:

$$\begin{aligned} & [2[n(n+1)/2] - n] / n \\ &= n \\ &= O(n) \end{aligned}$$

Standard Analysis Techniques

Constant time statements

Simplest case: $O(1)$ time statements

- Assignment statements of simple data types

```
int x = y;
```

- Arithmetic operations:

```
x = 5 * y + 4 - z;
```

- Array referencing:

```
A[j] = 5;
```

- Array assignment:

```
 $\forall j, A[j] = 5;$ 
```

- Most conditional tests:

```
if (x < 12) ...
```

Standard Analysis Techniques

Analyzing Loops

Any loop has two parts:

1. How many iterations are performed?
2. How many steps per iteration?

```
int sum = 0, j;  
for (j=0; j < N; j++)  
    sum = sum + j;
```

- Loop executes N times (0..N-1)
- 4 = $O(1)$ steps per iteration
- Total time is $N * O(1) = O(N*1) = O(N)$

Standard Analysis Techniques

Analyzing Loops (2)

What about this for-loop?

```
int sum = 0, j;  
for (j=0; j < 100; j++)  
    sum = sum + j;
```

- Loop executes 100 times
- 4 = $O(1)$ steps per iteration
- Total time is $100 * O(1) = O(100 * 1) = O(100) = O(1)$

PRODUCT RULE

Standard Analysis Techniques

Analyzing Loops (3)

What about `while`-loops?

Determine how many times the loop will be executed:

```
bool done = false;
int result = 1, n;
scanf("%d", &n);
while (!done) {
    result = result * n;
    n--;
    if (n <= 1) done = true;
}
```

Loop terminates when `done == true`, which happens after N iterations. Total time: $O(N)$

Standard Analysis Techniques

Nested Loops

Treat just like a single loop and evaluate each level of nesting as needed:

```
int j,k;  
for (j=0; j<N; j++)  
    for (k=N; k>0; k--)  
        sum += k+j;
```

Start with outer loop:

- How many iterations? N
- How much time per iteration? Need to evaluate inner loop

Inner loop uses $O(N)$ time

Total time is $N * O(N) = O(N*N) = O(N^2)$

Standard Analysis Techniques

Nested Loops (2)

What if the number of iterations of one loop depends on the counter of the other?

```
int j,k;  
for (j=0; j < N; j++)  
    for (k=0; k < j; k++)  
        sum += k+j;
```

Analyze inner and outer loop together:

- Number of iterations of the inner loop is:

$$0 + 1 + 2 + \dots + (N-1) = O(N^2)$$

Standard Analysis Techniques

Sequence of Statements

For a sequence of statements, compute their complexity functions individually and add them up

```
for (j=0; j < N; j++)  
    for (k =0; k < j; k++)  
        sum = sum + j*k;  
for (l=0; l < N; l++)  
    sum = sum -1;  
printf("sum is now %f", sum);
```

$\left. \begin{array}{l} \text{for (j=0; j < N; j++)} \\ \text{for (k =0; k < j; k++)} \\ \text{sum = sum + j*k;} \end{array} \right\} O(N^2)$
 $\left. \begin{array}{l} \text{for (l=0; l < N; l++)} \\ \text{sum = sum -1;} \end{array} \right\} O(N)$
 $\left. \begin{array}{l} \text{printf("sum is now \%f", sum);} \end{array} \right\} O(1)$

Total cost is $O(N^2) + O(N) + O(1) = O(N^2)$

SUM RULE

Standard Analysis Techniques

Digression

When doing Big-O analysis, we sometimes have to compute a series like:

$$1 + 2 + 3 + \dots + (N-1) + N$$

What is the complexity of this?

Remember Gauss:

$$\sum_{i=1}^n i = \frac{n * (n+1)}{2} = O\left(\frac{n^2 + n}{2}\right)$$

Standard Analysis Techniques

Conditional Statements

What about conditional statements such as

```
if (condition)
    statement1;
else
    statement2;
```

where `statement1` runs in $O(N)$ time and `statement2` runs in $O(N^2)$ time?

We use "worst case" complexity: among all inputs of size N , what is the maximum running time?

The analysis for the example above is $O(N^2)$

Fast machine Vs Fast Algorithm

Get a 10 times fast computer, that can do a job in 10^3 seconds for which the older machine took 10^4 seconds .

Comparing the performance of algorithms with time complexities $T(n)$ s of n , n^2 and 2^n (technically not an algorithm) for different problems on both the machines.

Question: Is it worth buying a 10 times fast machine?